

P04: Makers Makin' It, Act II -- The Sequel

PUSHEEN: EVAN KHOSH, MEGAN KWOK, ARTEMIS LEE, JOYCE LIN, SOPHIA CHI

DESIGN DOCUMENT

TARGET SHIP DATE: 2026-04-27

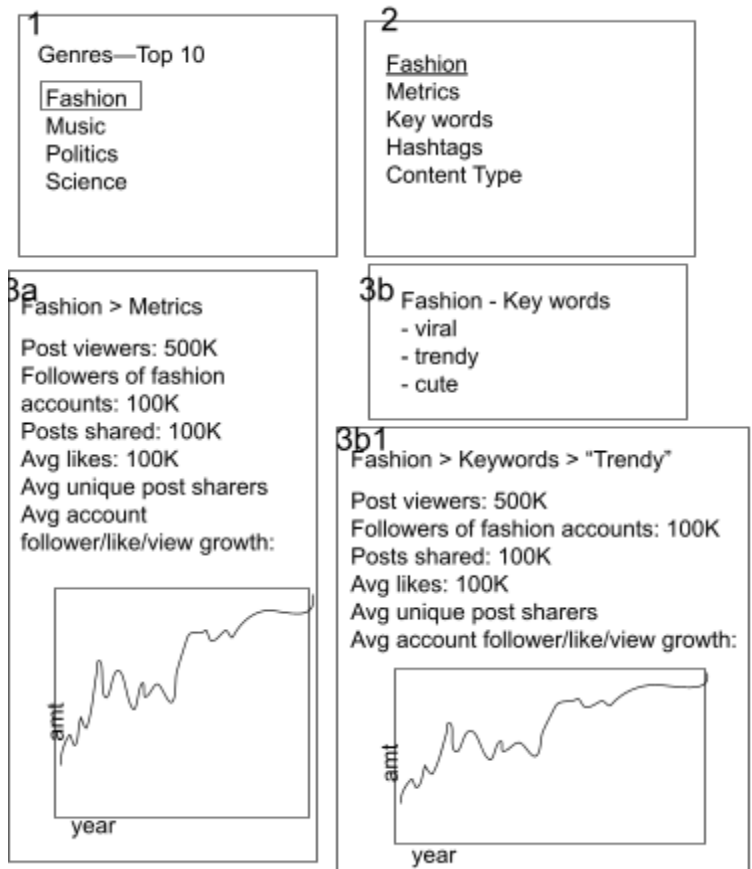
APP OVERVIEW:

Instagram data visualization

We will explore the types of posts and content (genres, styles, video-length, content format, amount of words in captions, hashtags, key words, username) that garners the most interaction and views (and what this says about how Instagram is encouraging influencers to post in a way that will keep Instagram users hooked onto the app).

The figure on the right is an example of how the app may function:

- The top most popular genres that trend on instagram are listed (graphic part 1)
- The data for this genre can be sorted by sections such as general metrics, key words, hashtags, etc (graphic part 2)
- In general metrics, the general overview of how this genre has done in terms of average post viewers, followers, posts shared, etc (graphic part 3a)
- In other sections such as key words (graphic part 3b), the key words can be listed and clicked into to show their metrics (graphic part 3b1)



ROLES:

Possible responsibilities: Database, Python, FEF (probably not much work needed for this one), HTML

1. MEGAN KWOK (PM)
2. EVAN KHOSH

3. ARTEMIS LEE
4. SOPHIA CHI
5. JOYCE LIN

PROGRAM COMPONENTS:

Language: Python

- `__init__.py`
 - Utilizes Flask to serve our app on our nginx servers as well as facilitates communication between our frontend and backend components (including other existing python files as well as javascript files).
- `math.py`
 - Will contain calculations necessary for processing our datasets in accordance with user inputs. The outputted data points will be fed back to `__init__.py` and subsequently displayed in our graphs.
- `db.py`
 - Will contain all MongoDB communications
 - Pulls out data in response to calls from functions involved in `math.py`,
 - storing of new data (gained from running our simulations)
 - and all necessary manipulations.

Language: Javascript

- `data.js`
 - Contains all code pertaining to the creation of data visualizations (line graphs, charts, etc.)

Framework: Flask

Our go-to framework, and the one we're most familiar with, served on Nginx. Will support the communication of information between pages as well as allow data to be fed into our chosen data visualization library.

DB: MongoDB

Document-oriented database that allows for increased flexibility when storing data, allowing for differing structures within collections as opposed to SQL's more rigid framework. This is our preferred method in data storage as it allows for easy handling of large datasets especially with its support of JSON which is essential for our project as it will allow us to utilize arrays for a group of hashtags and the like.

Structure: HTML

- *login.html*
 - Allows the user to log-in to a registered account by sending a POST request to `__init__.py` which will then pull the associated authentication info from our user database and compares it to that included in the user response.
 - Inaccurate info will prompt user to either reattempt their sign-in or register for an account
 - Accurate info will redirect the user to the landing page
- *register.html*
 - Allows the user to create an account by sending a POST request to `__init__.py` which will then query our database for any existing users that hold the same username.
 - If matches are detected, user is prompted to enter a new username
 - If no matches are detected, user is redirected to the landing page and the new user information is stored in the database.
- *landing.html*
 - Contains a short description of the purpose and function of our simulation (which essentially envisions the growth of a video posted with the selected parameters) as well as a start button that will redirect the user to `simulation.html`
 - POSSIBLE ADDITION: Buttons that allow user to essentially select the type of simulation that will occur
- *simulation.html*
 - By working with `__init__.py` and `data.js`, a simulation will be created and ran based upon user input on the scale and existence of certain metrics (user will choose to enable certain metrics such as video-length as well as then deciding on the length of the video or choosing to randomize it)
 - A 'BEGIN SIMULATION' button will be included which will essentially send a POST request to `__init__.py` which will then calculate and return info on predicted reach and viewer interactions. This will also be visualized in a graph created by our chosen data visualization library.

FEF SELECTION: TAILWIND

- We will make use of Tailwind as our front-end framework. Core elements from Tailwind that we'd like to use include classes and Tailwind's high customizability not offered in either Bootstrap and Foundations.

APIS IN USE

None

DATA VISUALIZATION LIBRARY SELECTION: [Chart.js](#)

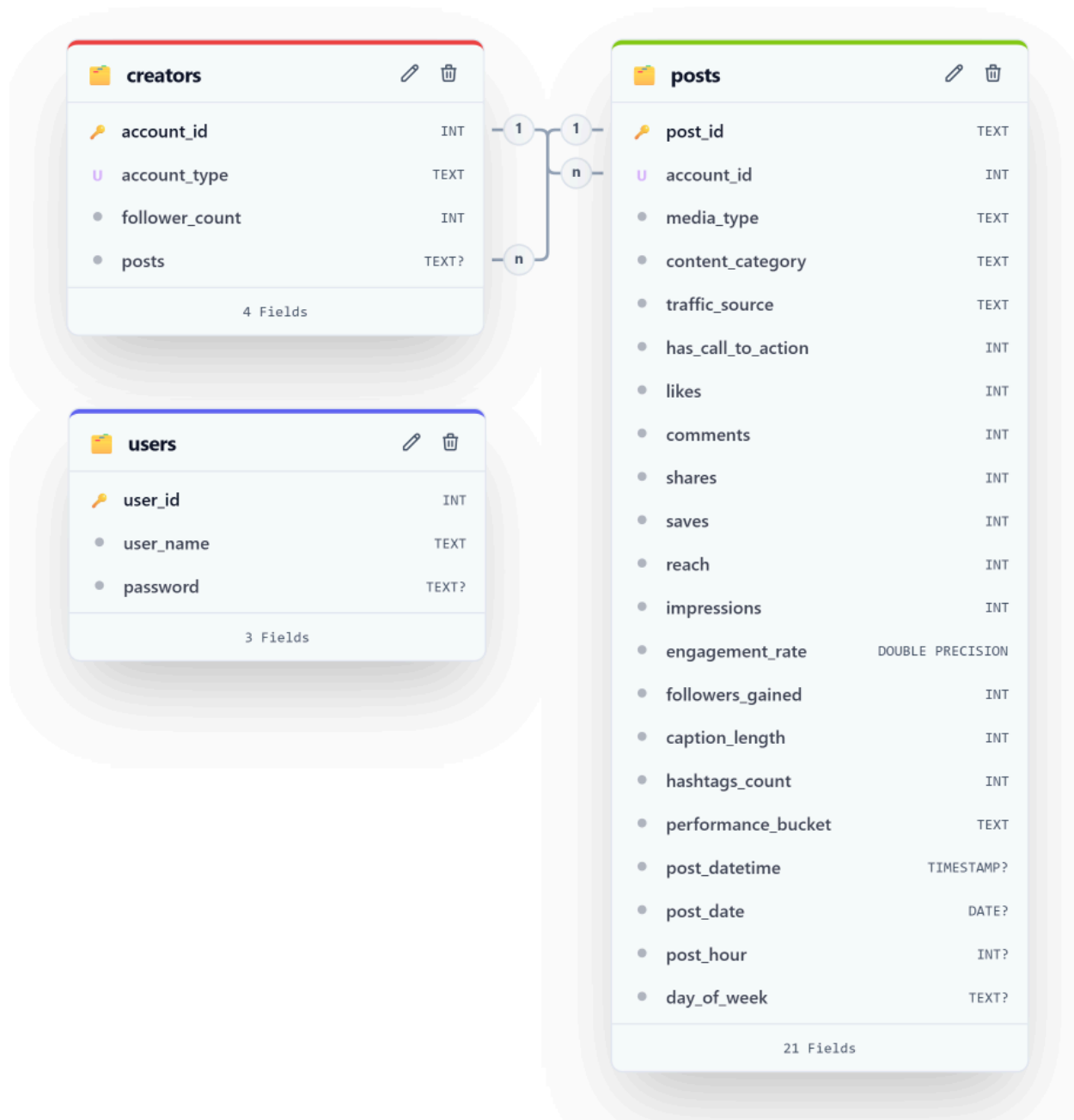
- We will likely be creating visualizations of our data and simulations through the usage of Chart.js. This library provides basic charts/graphs with a more simplistic appearance which is reflected in the corresponding sample code. As it both provides all that we would really need to bring our project to life as well as utilizes much shorter and nicer code, it is the better choice when compared to that of D3.js
- Chart.js also allows for animations/transitions as D3.js does, which allows for us to visualize and display our data in a much more dynamic way (rather than boring static images). The provided docs and sample code to such is, again, much easier to understand and utilize than D3.js.

TASKS/ASSIGNMENTS:

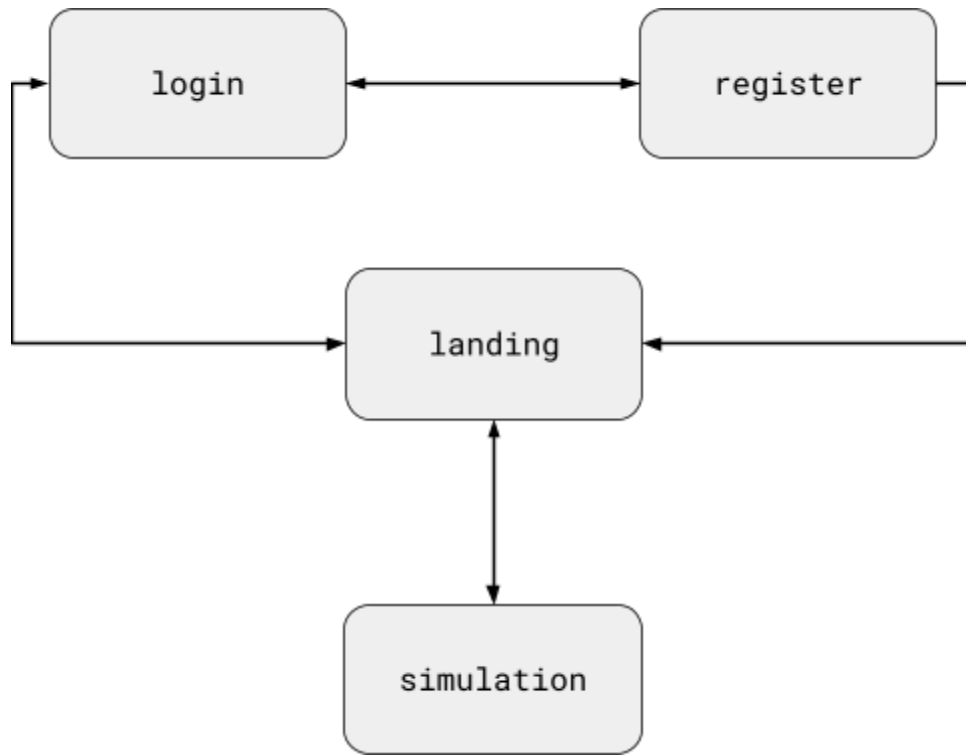
- ☐ Database Setup/Management (Evan)
 - ☐ Pull dataset from Kaggle
 - ☐ Set-up MongoDB database/server
- ☐ Backend/Python (Megan)
 - ☐ __init__.py - Main control for website functionality and communication between database and website
 - ☐ math.py - Functions to process data from database into more usable/functional values
 - ☐ db.py - Calls to database will be facilitated through pre-made functions in this file to prevent errors and bolster security
- ☐ Server Setup/Management (Joyce, Artemis)
 - ☐ Configure Flask, Nginx, & Unicorn
 - ☐ Ensure smooth connections between components
- ☐ Frontend/HTML (Sophia)
 - ☐ login.html - Log in user
 - ☐ register.html - Register user
 - ☐ landing.html - First page the user sees that describes the project and how to use + leads the user to other pages
 - ☐ simulation.html - Data visualization page where all the graphs/plots will be and data storytelling will take place
- ☐ Data Visualization (Evan, Megan)

- ☐ Solidify choice for data visualization method (Graph, Plot, etc.)
- ☐ Allow customizability for user (constraints & field types)
- ☐ FEF/Webpage Customization (Joyce, Sophia)
 - ☐ Make website clean and visually appealing
 - ☐ Add functionality through Tailwind buttons and more

DATABASE ORGANIZATION



SITE MAP



COMPONENT MAP

